Hiking Assistant - design report

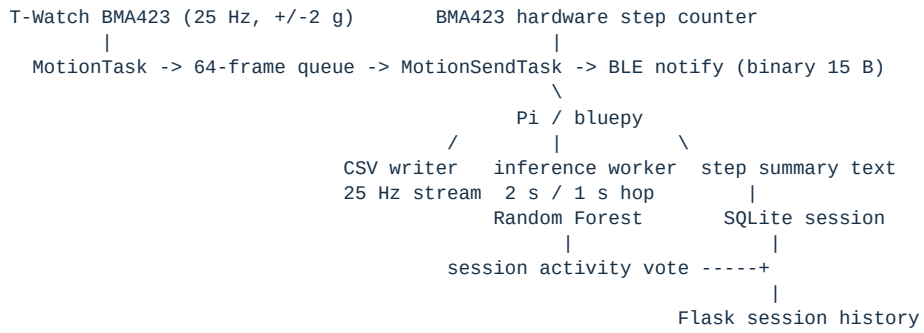
This report describes the architecture and implementation of the Hiking Assistant system.

1. Scope and implementation status

The system combines a TTGO T-Watch 2020 V3 (ESP32 and onboard BMA423) with a Raspberry Pi 3B+ hub. The watch counts steps, samples three-axis wrist acceleration during an active session, and sends BLE notifications. The Pi saves received samples to CSV, runs a lightweight motion classifier, stores a session summary in SQLite, and displays past sessions in a local Flask page.

The firmware compiles with Arduino ESP32 core 2.0.17 and LilyGO TWatch Library 1.4.2. Packet parsing, replay, database behavior, and training were exercised without hardware. BLE interoperability, BMA423 scaling and orientation, runtime timing, battery life, and target-device classification remain unverified. The classifier is a prototype, and the web login is suitable only for a trusted local development environment.

2. System boundary and data flow



The raw stream and the session summary have different purposes. CSV is the analysis and retraining record; SQLite stores only the derived session fields. The watch has no persistent motion buffer. A notification sent while disconnected is lost, and a reconnect starts a new receiver file.

3. Watch software design

The Arduino sketch is `Watch/hikingassistant_RTOS/hikingassistant_RTOS.ino`; implementation is in `hikingassistant.cpp`, with UI and event code in `inc/` and `src/`.

`TouchTask` handles touch and application events. States are `APP_IDLE`, `APP_COUNTING`, `APP_SESSION_FINISHED`, and `APP_UPLOADING`. `StepTask` handles the BMA423 step-count interrupt and refreshes the visible count. `MotionTask` samples acceleration during `APP_COUNTING` at a nominal 40 ms period; `MotionSendTask` takes packets from a bounded FreeRTOS queue. `BleTask` restarts advertising after disconnect. Mutexes protect shared state, BMA/touch access, and the BLE transmit characteristic. The motion producer does not block on a full queue: it increments a cumulative drop counter. No SD/flash motion storage, replay, or watch-side classification exists.

The BMA423 is configured at 25 Hz, +/-2 g, continuous mode. The library `getAccel()` values are converted from nominal 1024 LSB/g to integer milligravity. The hardware step counter is distinct from the Python motion classifier. The code does not use the classifier to calculate steps or calories. Step distance uses `steps * 0.0008 km`; calories use the inherited fixed heuristic in `RPi/hike.py`. These are estimates, not calibrated measurements.

4. BLE application protocol

The firmware exposes a Nordic UART-style service: service `6E400001-B5A3-F393-E0A9-E50E24DCCA9E`, Pi-to-watch RX characteristic `6E400002-B5A3-F393-E0A9-E50E24DCCA9E`, and watch-to-Pi TX notification characteristic `6E400003-B5A3-F393-E0A9-E50E24DCCA9E`. The Pi discovers these UUIDs, enables the TX CCCD, sends `c` on connection and `r` after storing an uploaded summary. The watch sends the final step count as short ASCII text after the user chooses UPLOAD.

Each motion notification is exactly 15 bytes, little endian (`<BHIhhhH`):

Offset	Type	Meaning
0	uint8	protocol type/version `0x11`
1	uint16	sequence, wraps after 65535
3	uint32	watch uptime in milliseconds, wraps after 2^32-1
7, 9, 11	int16 each	X, Y, Z acceleration in mg
13	uint16	cumulative sensor/producer queue drops

The 15-byte payload fits a default BLE ATT MTU of 23 bytes. The sequence restarts at zero when sampling resumes for a new session. The Pi compares sequence, watch time, and cumulative drops; a detected gap discards the partially built inference window. It records the received sample regardless. Counters wrap, and there is no retransmission or cryptographic integrity check. The `watch_time_ms` field describes watch uptime, not wall-clock time.

5. Pi receive, storage, and inference

`RPi/receiver.py` initializes the database, loads a trusted model if present, and reconnects the BLE client after a lost link. `RPi/bt.py` decodes binary frames through `stream_protocol.py`, dispatches recording and inference to bounded worker queues, and parses the separate ASCII session upload. A new `RPi/recordings/*.csv` is created for each receiver connection that sends motion samples. CSV columns are `sequence,watch_time_ms,x_g,y_g,z_g,watch_dropped,received_at_utc,gap_before`. `received_at_utc` is Pi wall-clock receipt time for annotation; `gap_before` records `sequence`, `watch_time`, or `watch_dropped` anomalies before the row. Replay uses the same discontinuity criteria to avoid feature windows spanning missing samples. Queue overflow is logged. CSV is flushed periodically and on a clean shutdown; a crash can lose the last buffered rows.

`RPi/features.py` computes 48 window features from the three axes and magnitude. A model artifact specifies 25 Hz, 50 samples per window (2 seconds), and a 25-sample hop (1 second). `activity_rec.py` predicts `idle`, `walking`, `climbing`, or `running` on the Pi. `climbing` means the training proxy `upstairs`; hill ascent was not separately trained. `running` uses source `jogging`. On a gap, only the partial feature window resets. The session activity mode is the most frequent prediction, and the distribution is a share of prediction windows, not measured time. If no compatible model is available, the session label is `unknown`.

`RPi/db.py` stores sessions in SQLite (`RPi/sessions.db`) with `session_id`, `km`, `steps`, `burnt_kcal`, `session_user`, `activity_mode`, and JSON `activity_distribution`. It supports migration when activity columns are absent. `passwords` stores a plaintext username/password pair. `RPi/wserver.py` serves login and newest-first session history on `127.0.0.1:5001` by default. The login is session-based; there is no role-based access control, account registration, or public network hardening. The model-status API reports only file presence, not model validity.

6. Model provenance and reproducibility

`RPi/train_public.py` consumes the public SELFBack dataset, `raw/6_class/wrist50/`. These are Axivity AX3 wrist accelerometer recordings at 100 Hz, not BMA423 measurements. `sitting` and `standing` map to `idle`, `walking` to `walking`, `upstairs` to `climbing`, and `jogging` to `running`; `downstairs` is excluded. The loader filters and resamples to 25 Hz, creates 2-second windows with a 1-second hop, extracts the same 48 features as live inference, and trains a scikit-learn Random Forest. Subjects are separated across train and test. The saved pickle records model and feature metadata; only load an artifact generated by a trusted source, with the matching scikit-learn version.

The held-out public-data run is summarized in `MODEL_EVALUATION.md`. Its overall window accuracy was 0.852 and macro F1 was 0.818 for one subject split. Walking and upstairs are frequently confused. These are public-data results, not T-Watch field accuracy. No claim of reliable hill ascent recognition, device calibration, or deployment readiness follows from them.

7. Failure modes and operating constraints

- BLE disconnect: the receiver closes the current CSV, discards its partial inference window, and retries connection; unsent watch frames are not recovered.
- Producer queue, CSV queue, or inference queue overflow: samples can be lost. Sequence numbers and logs expose some gaps; recording may still have local loss after BLE receipt.
- Model absent or incompatible: recording and step summaries continue, activity is `unknown`.
- Disk error: CSV writing logs failure; no alternative durable storage exists.
- Power loss: SQLite and any already flushed CSV survive according to the filesystem state; the watch does not retain an unfinished session.
- Security: BLE and local web are prototype-only. Default credentials and plaintext password storage require redesign before any untrusted-network use.

8. Verification and remaining field work

Automated tests cover the packet codec, units and endianness, training/live feature agreement, window timing, gap reset, CSV flush, and SQLite migration/session order. Run `python -m unittest discover -s tests -v` from the repository root with the Python requirements installed. The Arduino sketch was compile-checked; neither watch nor Pi hardware was available for end-to-end testing.

With hardware, validate static gravity (~1 g), axis orientation, nominal 25 Hz timestamps and jitter, BLE characteristic subscription, notification throughput, disconnect/reconnect, full and empty queue behavior, step counter reset, on-device power consumption, and long-running disk use. Collect labeled T-Watch sessions across wearers and conditions before estimating target-device accuracy or changing the model.

9. Key files

- Watch: `Watch/hikingassistant_RTOS/hikingassistant.cpp`, `inc/events.h`, `src/events.cpp`, `inc/ui.h`, `src/ui.cpp`.
- Pi transport: `RPi/bt.py`, `RPi/stream_protocol.py`, `RPi/receiver.py`.
- Inference and training: `RPi/features.py`, `RPi/activity_rec.py`, `RPi/train_public.py`, `RPi/replay.py`, `RPi/export_public_sample.py`.
- Persistence and web: `RPi/db.py`, `RPi/hike.py`, `RPi/wserver.py`, `RPi/home.html`, `RPi/login.html`.
- Tests and evidence: `tests/test_stream.py`, `MODEL_EVALUATION.md`.